

**Міністерство освіти і науки України**
**Національний технічний університет України**
**"Київський політехнічний інститут імені Ігоря Сікорського"**
Фізико-технічний інститут

# Аналіз кіберінцидентів
Практична робота №1

Виконали:
студенти гр. ФБ-62мп
Маврикін Едуард
Смірнова Наталія

Київ–2026

**Хід роботи:**

**Тема: Логістична регресія в системах машинного навчання**

Мета: використовуючи модель логістичної регресії, виконати класифікацію програмного забезпечення на шкідливе та безпечне на даних датасету Malware Executable Detection.

## 1. Теоретичні відомості

Класифікація шкідливого ПЗ є задачею машинного навчання з учителем (supervised learning): модель навчається на прикладах з відомими мітками класів і узагальнює їх на нові, раніше невідомі файли. На відміну від статичного порівняння сигнатур, машинне навчання дає нечітке порівняння об'єктів, автоматичне визначення ваги (значущості) ознак та адаптованість до нових варіантів шкідливого ПЗ.

Логістична регресія оцінює ймовірність належності об'єкта до класу 1:
$p(Y=1 \mid x) = \Lambda(\theta^T x) = 1 / (1 + e^{-\theta^T x}), \quad \theta^T x = \theta 0 + \theta 1 x1 + \ldots + \theta n x n$
Об'єкт відноситься до класу 1, якщо оцінка ймовірності перевищує поріг відсікання k = 0.5. Знак і величина коефіцієнта θі показують, як ознака хі впливає на шанси (odds) віднесення до шкідливого класу.

Якість моделі перевіряється на тестовій вибірці, яка не брала участі у навчанні, оскільки модель, що добре працює на тренувальних даних, може мати перепідгонку (overfitting) і погано узагальнюватись.

## 2. Вихідні дані

Датасет Malware Executable Detection (Kaggle, також є в UCI) містить 373 виконувані файли Windows: 301 шкідливий (malicious) та 72 безпечні (non-malicious). Кожен файл описано 531 бінарною ознакою F_1 … F_531, отриманою статичним аналізом: гібридні ознаки з бінарного шістнадцяткового коду та викликів DLL. Пропущених значень немає.

Особливості даних: вибірка незбалансована (≈ 81 % / 19 %), а кількість ознак (531) більша за кількість прикладів (373). Тому структуру моделі необхідно спрощувати, залишаючи найбільш істотні ознаки.

## 3. Побудова моделі

Позитивною подією обрано клас malicious (1), негативною – non-malicious (0). Модель побудовано мовою Python (pandas, scikit-learn, matplotlib):
– вибірку розділено на навчальну (75 %) та тестову (25 %) частини зі стратифікацією;
– ознаки з дисперсією менше 0.01 (майже константні) відкинуто – залишилось 464;
– LogisticRegression навчено з class_weight="balanced" для компенсації дисбалансу та регуляризацією C=0.1 для запобігання перенавчанню;
– додатково виконано 5-кратну стратифіковану крос-валідацію.

Код програми:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.feature_selection import VarianceThreshold
from sklearn.model_selection import train_test_split, cross_val_score,
StratifiedKFold
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    roc_auc_score,
    RocCurveDisplay,
    ConfusionMatrixDisplay,
)

df = pd.read_csv("uci_malware_detection.csv")
X = df.drop(columns="Label")
y = (df["Label"] == "malicious").astype(int)
print("Розмір:", X.shape)
print(df["Label"].value_counts())

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42
)

model = make_pipeline(
    VarianceThreshold(0.01),
    LogisticRegression(C=0.1, max_iter=1000, class_weight="balanced"),
)
model.fit(X_train, y_train)
selected = X.columns[model[0].get_support()]
print("Ознак після відбору:", len(selected))

y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

print("Accuracy:", round(accuracy_score(y_test, y_pred), 4))
```

```
print("ROC AUC:", round(roc_auc_score(y_test, y_prob), 4))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=["non-malicious",
"malicious"]))

cv = StratifiedKFold(5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=cv, scoring="f1")
print("F1 5-fold CV:", scores.round(3), "середнє:", round(scores.mean(),
3))

coef = pd.Series(model[-1].coef_[0], index=selected).sort_values()
print("Intercept θ0:", round(model[-1].intercept_[0], 3))
print("Топ-5 ознак malicious:\n", coef.tail(5)[::-1].round(3))
print("Топ-5 ознак non-malicious:\n", coef.head(5).round(3))

fig, ax = plt.subplots(1, 2, figsize=(11, 4.5))
ConfusionMatrixDisplay.from_predictions(
    y_test, y_pred, display_labels=["non-malicious", "malicious"],
ax=ax[0], colorbar=False
)
RocCurveDisplay.from_predictions(y_test, y_prob, ax=ax[1])
plt.tight_layout()
plt.savefig("results.png", dpi=120)

top = pd.concat([coef.head(10), coef.tail(10)])
plt.figure(figsize=(8, 6))
top.plot.barh(color=["tab:green"] * 10 + ["tab:red"] * 10)
plt.xlabel("Коефіцієнт")
plt.tight_layout()
plt.savefig("coef.png", dpi=120)
```

## 4. Результати

Результат виконання програми:

```
Розмір: (373, 531)
Label
malicious        301
non-malicious     72
Name: count, dtype: int64
Ознак після відбору: 464
Accuracy: 1.0
ROC AUC: 1.0
[[18  0]
 [ 0 76]]
              precision    recall  f1-score   support

non-malicious       1.00      1.00      1.00        18
    malicious       1.00      1.00      1.00        76

     accuracy                           1.00        94
    macro avg       1.00      1.00      1.00        94
 weighted avg       1.00      1.00      1.00        94
```

```
F1 5-fold CV: [1.    0.992 1.    0.992 1.   ] середнє: 0.997
Intercept θ0: -1.939
Топ-5 ознак malicious:
 F_20     0.309
F_53     0.212
F_250    0.210
F_526    0.193
F_530    0.182
dtype: float64
Топ-5 ознак non-malicious:
 F_19    -0.313
F_69    -0.280
F_140   -0.195
F_50    -0.181
F_66    -0.177
dtype: float64
```

На тестовій вибірці (94 файли) модель класифікувала всі об'єкти правильно: accuracy = 1.0, ROC AUC = 1.0, помилок першого та другого роду немає (рис.1).

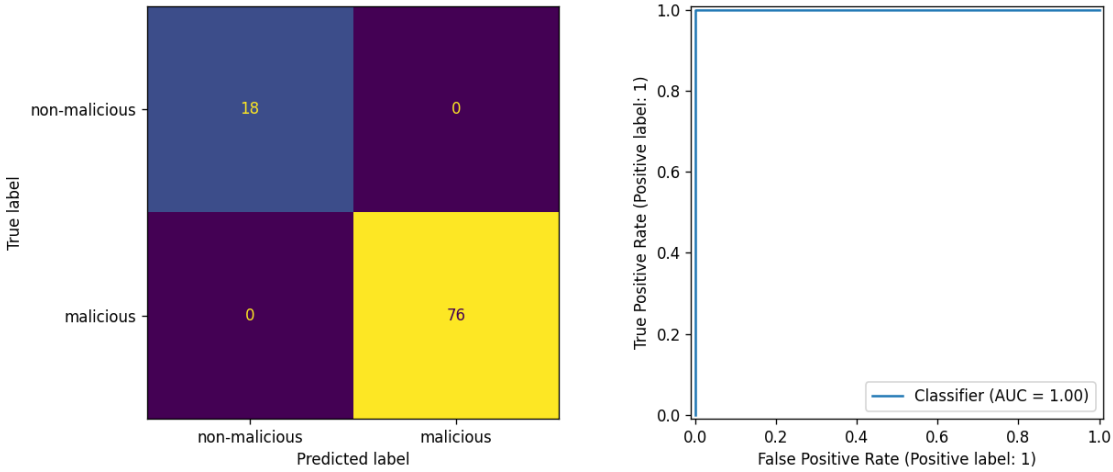



рис.1

Через малий обсяг тестової вибірки результат перевірено крос-валідацією: F1 по 5 блоках становить 0.992–1.0, середнє 0.997. Отже, висока точність не є наслідком вдалого розбиття, і модель стабільно узагальнює дані.

Коефіцієнти моделі показують внесок кожної ознаки: додатні збільшують ймовірність класу malicious, від'ємні – зменшують (рис.2). Назви ознак у датасеті знеособлені, тому інтерпретація можлива лише на рівні номерів ознак.

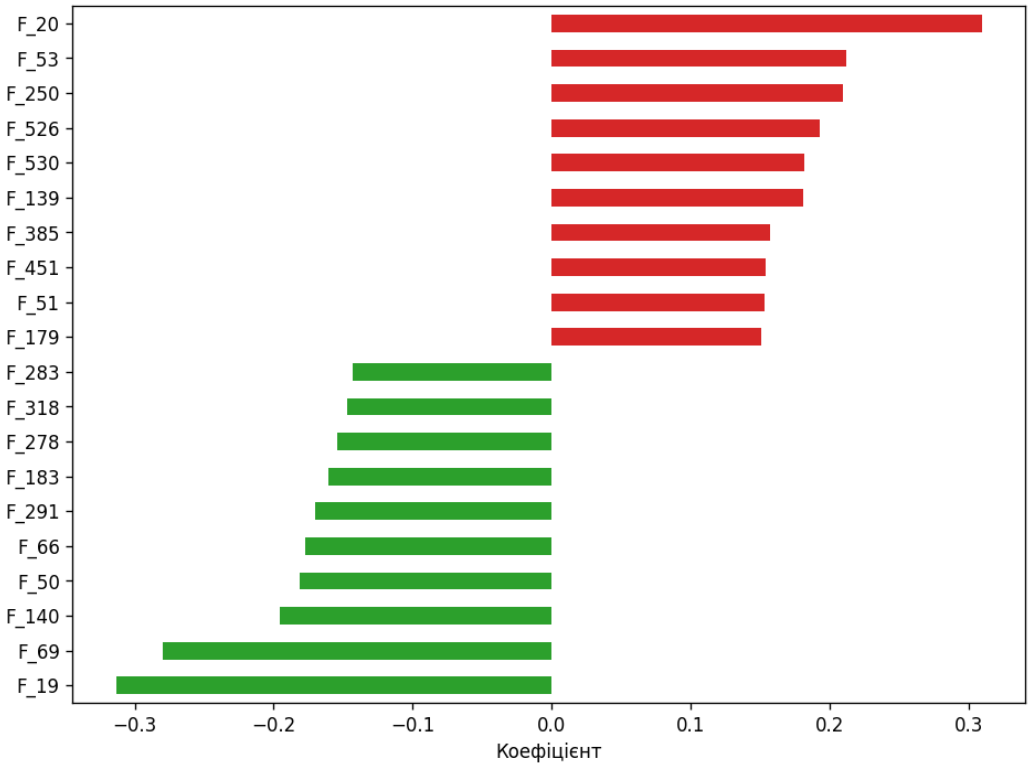



рис.2

**5. Висновок**

У ході роботи побудовано модель логістичної регресії для класифікації виконуваних файлів Windows на шкідливі та безпечні за ознаками статичного аналізу (шістнадцятковий код + виклики DLL). Модель показала accuracy 1.0 на тестовій вибірці та середнє F1 0.997 на крос-валідації.

Оскільки ознак більше, ніж прикладів, було застосовано відбір ознак і регуляризацію, а дисбаланс класів враховано вагами. Логістична регресія виявилась простим, швидким та інтерпретованим методом, придатним для попереднього виявлення шкідливого ПЗ ще до його запуску.